06. 에디터와 소프트웨어 설치

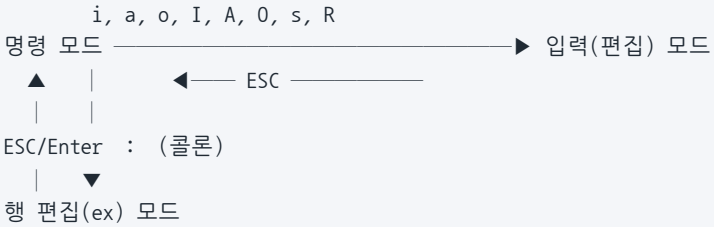
1. 에디터 종류

에디터	개발자	특징
vi	**빌 조이(Bill Joy)**	UNIX 표준 에디터, 모드 기반
vim	**브람 무레나르(Bram Moolenaar)**	vi improved — 구문 강조, 다중 undo
emacs	**리처드 스톨만(Richard Stallman)**	매우 강력, Lisp 확장, 통합 환경
pico	아보일 카사디아스	간단, Pine 메일러 부속
nano	-	pico의 GNU 대체판, 초보자 친화
gedit	-	GNOME 계열 GUI 에디터
kwrite / kate	-	KDE 계열 GUI 에디터
ed / ex	켄 톰슨	라인 단위 에디터

출제 포인트

- **vi = 빌 조이**, **vim = 브람 무레나르**, **emacs = 리처드 스톨만** (이 3개 조합이 단골)
- `nano` 는 `pico` 를 대체하기 위해 만든 GNU 에디터

2. vi 에디터 — 3가지 모드



모드	진입	역할
명령 모드(Command)	기본 / ESC	커서 이동, 삭제, 복사
입력 모드(Insert)	`i` , `a` , `o` 등	실제 텍스트 입력
행 편집 모드(ex/Last line)	`:`	저장, 종료, 치환, 설정

3. vi 입력 모드 진입 키 (구분 중요)

키	동작
`i`	커서 **앞**에 삽입 (insert)
`I`	**행의 맨 앞**에 삽입
`a`	커서 **뒤**에 삽입 (append)
`A`	**행의 맨 끝**에 삽입

키	동작
`o`	커서 **아래**에 새 행 추가 (open)
`O`	커서 **위**에 새 행 추가
`s`	커서 위치 한 글자 삭제 후 입력
`S`	현재 행 전체 삭제 후 입력
`R`	덮어쓰기(Replace) 모드
`cw`	단어 변경(change word)

---

4. vi 커서 이동

키	동작
`h` `j` `k` `l`	좌 / 하 / 상 / 우
`w`	다음 단어의 첫 글자
`b`	이전 단어의 첫 글자
`e`	단어의 끝
`0` (숫자 영)	**행의 처음**
`^`	행의 첫 글자(공백 제외)
`$`	**행의 끝**
`gg`	**문서 맨 처음**
`G`	**문서 맨 끝**
`nG` 또는 `:n`	n번째 행으로 이동
`H` `M` `L`	화면 상단 / 중간 / 하단
`Ctrl+f` / `Ctrl+b`	한 화면 앞으로 / 뒤로
`Ctrl+d` / `Ctrl+u`	반 화면 아래 / 위
`%`	짝이 되는 괄호로 이동

---

5. vi 삭제·복사·붙여넣기

키	동작
`x`	커서 위치 한 글자 삭제
`X`	커서 앞 한 글자 삭제
`nx`	n개 문자 삭제
`dd`	**현재 행 삭제(잘라내기) ★**
`ndd`	n개 행 삭제 (`3dd` = 3행 삭제)
`dw`	단어 삭제
`d$` 또는 `D`	커서부터 행 끝까지 삭제
`d0`	커서부터 행 처음까지 삭제

키	동작
`dG`	커서부터 문서 끝까지 삭제
`yy`	**현재 행 복사(yank) ★**
`nyy`	n개 행 복사
`yw`	단어 복사
`p`	**커서 아래(뒤)에 붙여넣기**
`P`	**커서 위(앞)에 붙여넣기**
`u`	**undo (실행 취소)**
`U`	현재 행의 변경 취소
`Ctrl+r`	redo (다시 실행)
`.`	**직전 명령 반복**
`J`	현재 행과 다음 행 합치기
`~`	대소문자 전환

---

6. vi 검색과 치환

```
/pattern      아래 방향 검색
?pattern      위 방향 검색
n             다음 검색 결과
N             반대 방향 다음 결과
```

치환 (:s 명령 — 시험 단골 ★★)

```
:s/old/new/           현재 행의 첫 번째만 치환
:s/old/new/g          현재 행 전체 치환
:%s/old/new/g         ★ 문서 전체 치환 (% = 전체 행)
:%s/old/new/gc        전체 치환 + 각각 확인(confirm)
:1,10s/old/new/g      1~10행만 치환
:.,$s/old/new/g       현재 행부터 끝까지
:g/pattern/d          pattern이 포함된 행 전체 삭제
```

기호	의미
`%`	문서 전체 행
`.`	현재 행
`$`	마지막 행
`g`	global — 행 내 모든 일치
`c`	confirm — 치환 시 확인
`i`	대소문자 무시

---

7. vi 저장과 종료

명령	동작
`:w`	저장(write)
`:w newfile`	다른 이름으로 저장
`:q`	종료(quit) — 변경사항 있으면 거부
`:wq`	**저장 후 종료 ★**
`:wq!`	강제 저장 후 종료
`ZZ`	`:wq` 와 동일 (명령 모드에서)
`:q!`	**저장하지 않고 강제 종료 ★**
`:x`	변경된 경우만 저장 후 종료
`:e!`	저장 후 마지막 상태로 되돌리기
`:r file`	다른 파일 내용 삽입(read)
`:r !date`	명령 실행 결과 삽입
`:!command`	쉘 명령 실행
`:sh`	임시로 쉘로 나가기 (`exit` 로 복귀)

---

8. vi 환경 설정

```
:set nu          행 번호 표시 (number)
:set nonu        행 번호 제거
:set ai          자동 들여쓰기 (autoindent)
:set ts=4        탭 크기 4 (tabstop)
:set sw=4        들여쓰기 폭 (shiftwidth)
:set ic          대소문자 무시 검색 (ignorecase)
:set hls         검색 결과 강조 (hlsearch)
:set list        비출력 문자 표시
:set showmode    현재 모드 표시
:set ruler       커서 위치 표시
:set all         모든 설정 보기
```

영구 적용: `~/.vimrc` (vim) 또는 `~/.exrc` (vi)

```
# ~/.vimrc 예
set nu
set ai
set ts=4
syntax on
```

vi 복구 (출제 포인트)

비정상 종료 시 스왑 파일(`.filename.swp`)이 생성됩니다.

```
vi -r filename       # 복구 모드로 열기
```

---

9. 패키지 관리 — 개요

계열	저수준(단일 패키지)	고수준(의존성 자동 해결)
Red Hat 계열 (`.rpm`)	`rpm`	`yum` → `dnf`
Debian 계열 (`.deb`)	`dpkg`	`apt-get` , `apt` , `aptitude`
Slackware (`.tgz`)	`installpkg`	`slackpkg`
SUSE (`.rpm`)	`rpm`	`zypper`
Arch	`pacman`	`pacman`

핵심 차이 (시험 단골)

- `rpm` / `dpkg` **는 의존성을 자동 해결하지 못합니다.** 의존 패키지가 없으면 오류.
- `yum` / `dnf` / `apt` **는 저장소(repository)에서 의존성까지 자동 설치합니다.**

---

10. rpm (Red Hat Package Manager)

패키지 파일명 규칙

```
httpd-2.4.6-97.el7.centos.x86_64.rpm
  |       |      |        |          └──── 아키텍처 (x86_64, i686, noarch, src)
  |       |      |        └─────── 배포판(el7 = Enterprise Linux 7)
  |       |      └──────── 릴리스 번호
  |       └─────── 버전
  └──────── 패키지 이름
```

설치·삭제·업그레이드

```
rpm -i pkg.rpm          # 설치(install)
rpm -ivh pkg.rpm        # ★ 설치 + 상세 + 진행바 (가장 많이 씀)
rpm -U pkg.rpm          # 업그레이드 (없으면 새로 설치)
rpm -Uvh pkg.rpm        # ★
rpm -F pkg.rpm          # freshen — 기존에 설치된 것만 갱신
rpm -e pkg              # 삭제(erase) — 패키지명만 씀 (.rpm 아님!)
rpm -e --nodeps pkg     # 의존성 무시 삭제
rpm -ivh --force pkg    # 강제 설치
rpm -ivh --test pkg     # 실제 설치 없이 테스트
```

옵션	의미
`-i`	install
`-U`	Upgrade (없으면 설치)
`-F`	Freshen (있을 때만 갱신)
`-e`	erase (삭제)
`-v`	verbose
`-h`	hash — `#` 진행 표시
`--nodeps`	의존성 무시
`--force`	강제

옵션	의미
`--test`	테스트만

질의(query) — `-q` 계열 ★

```
rpm -qa                # 설치된 전체 패키지 목록 (all)
rpm -qa | grep httpd   # 특정 패키지 설치 확인
rpm -qi httpd          # 패키지 정보 (info)
rpm -ql httpd          # 패키지에 포함된 파일 목록 (list) ★
rpm -qd httpd          # 문서 파일 목록
rpm -qc httpd          # 설정 파일 목록
rpm -qf /etc/httpd/conf/httpd.conf   # ★ 특정 파일이 어느 패키지 소속인지 (file)
rpm -qR httpd          # 의존성 확인 (Requires)
rpm -qp pkg.rpm -i     # 설치 전 rpm 파일 조회 (package)
rpm -qpl pkg.rpm       # 설치 전 파일 목록 확인
rpm -V httpd           # 패키지 검증 (verify) — 변경된 파일 확인
rpm -Va                # 전체 검증
rpm --import KEY       # GPG 키 등록
```

출제 포인트

- **`rpm -qf 파일경로`** = "이 파일은 어느 패키지에 속하는가?" → 매우 자주 출제
- **`rpm -ql 패키지`** = "이 패키지는 어떤 파일을 설치했는가?"
- 삭제할 때는 **패키지명**(예: `httpd`), 설치할 때는 **파일명**(예: `httpd-2.4.rpm`)

---

11. yum / dnf (Red Hat 계열 고수준)

```
yum install httpd          # 설치 (의존성 자동)
yum -y install httpd       # 확인 없이 설치
yum remove httpd           # 삭제 (= erase)
yum update                 # 전체 업데이트
yum update httpd           # 특정 패키지 업데이트
yum check-update           # 업데이트 가능 목록
yum list                   # 전체 패키지 목록
yum list installed         # 설치된 것만
yum list available         # 설치 가능한 것
yum info httpd             # 정보
yum search web             # 이름 · 설명 검색
yum provides */httpd.conf  # ★ 특정 파일을 제공하는 패키지 찾기
yum deplist httpd          # 의존성 목록
yum grouplist              # 패키지 그룹 목록
yum groupinstall "Development Tools"   # 그룹 설치 ★
yum groupremove "..."      # 그룹 삭제
yum clean all              # 캐시 삭제
yum repolist               # 저장소 목록
yum history                # 작업 이력
yum localinstall pkg.rpm   # 로컬 rpm을 의존성 해결하며 설치
```

- 저장소 설정 파일: **`/etc/yum.repos.d/*.repo`**
- 주 설정 파일: `/etc/yum.conf`
- **dnf**는 yum의 차세대 버전으로 명령 체계가 거의 동일합니다(RHEL 8+ 기본).

```
dnf install httpd
dnf group install "Development Tools"
```

---

12. dpkg / apt (Debian 계열)

```
# dpkg — 저수준
dpkg -i pkg.deb            # 설치
dpkg -r pkg                # 삭제 (설정 파일 유지)
dpkg -P pkg                # 완전 삭제 (purge)
dpkg -l                    # 설치된 패키지 목록
dpkg -L pkg                # 패키지의 파일 목록
dpkg -S /etc/file          # 파일이 속한 패키지 검색
dpkg -s pkg                # 상태 · 정보
dpkg --info pkg.deb        # deb 파일 정보

# apt / apt-get — 고수준
apt update                 # 패키지 목록 갱신 (저장소 정보) ★
apt upgrade                # 설치된 패키지 업그레이드
apt full-upgrade           # 의존성 변경 포함 업그레이드
apt install nginx          # 설치
apt remove nginx           # 삭제
apt purge nginx            # 설정까지 삭제
apt autoremove             # 불필요한 의존 패키지 제거
apt search nginx           # 검색
apt show nginx             # 정보
apt list --installed       # 설치 목록
apt-cache search nginx     # 검색(구 방식)
apt-cache depends nginx    # 의존성
```

- 저장소 설정: **`/etc/apt/sources.list`**, `/etc/apt/sources.list.d/`
- ⚠️ 혼동 주의: `yum update`는 **패키지 업그레이드**지만, `apt update`는 **목록 갱신**입니다. 실제 업그레이드는 `apt upgrade`.

---

13. 소스 컴파일 설치 (Tarball)

표준 3단계 (순서 암기 ★★★)

```
tar zxvf pkg-1.0.tar.gz       # 1) 압축 해제
cd pkg-1.0
./configure --prefix=/usr/local/pkg   # 2) 환경 설정 (Makefile 생성)
make                                  # 3) 컴파일
make install                          # 4) 설치
```

단계	역할
`./configure`	시스템 환경 검사 → **Makefile 생성**. `--prefix`로 설치 경로 지정
`make`	Makefile에 따라 **소스 컴파일**
`make install`	컴파일된 결과를 지정 경로에 **복사·설치**
`make clean`	컴파일 중간 파일 삭제
`make uninstall`	설치 제거 (지원하는 경우)

출제 포인트

- **순서:** `configure` → `make` → `make install`
- `./configure`가 만드는 파일은 **Makefile**
- 설치 경로 지정 옵션은 `--prefix`
- 컴파일에는 gcc, make, 개발 헤더 패키지가 필요 → `yum groupinstall "Development Tools"`

관련 도구

도구	역할
`gcc`	C 컴파일러
`g++`	C++ 컴파일러
`make`	빌드 자동화
`cmake`	크로스 플랫폼 빌드 설정
`ldd`	실행 파일의 공유 라이브러리 의존성 확인
`ldconfig`	공유 라이브러리 캐시 갱신 (`/etc/ld.so.conf`)
`ar`	정적 라이브러리(.a) 생성

---

14. 마무리 체크 문제

1. vi, vim, emacs의 개발자? → **빌 조이 / 브람 무레나르 / 리처드 스톨만**
2. 커서 아래에 새 행을 만들며 입력 모드로? → `o`
3. 현재 행을 삭제하는 vi 명령? → `dd`
4. 3개 행을 복사하는 명령? → `3yy`
5. 문서 전체에서 문자열을 모두 치환? → `:%s/old/new/g`
6. 저장하지 않고 강제 종료? → `:q!`
7. vi에서 행 번호를 표시하는 설정? → `:set nu`
8. vim 설정을 영구 저장하는 파일? → `~/.vimrc`
9. rpm으로 설치된 전체 패키지 목록? → `rpm -qa`
10. 특정 파일이 어느 패키지에 속하는지 확인? → `rpm -qf 파일경로`
11. rpm에서 패키지에 포함된 파일 목록? → `rpm -ql 패키지명`
12. rpm 삭제 옵션? → `-e`
13. rpm이 의존성을 자동 해결하는가? → **아니오** (yum/dnf가 해결)
14. yum 저장소 설정 파일 위치? → `/etc/yum.repos.d/*.repo`
15. apt에서 저장소 목록만 갱신하는 명령? → `apt update`
16. 소스 컴파일 설치 3단계 순서? → `./configure` → `make` → `make install`
17. `./configure`가 생성하는 파일? → **Makefile**
18. 설치 경로를 지정하는 configure 옵션? → `--prefix`